Limpieza y Transformación de Datos de Empleados utilizando SQL

Presentado por:

Fanny Tlecuitl

Contenido

En este proyecto se llevó a cabo un proceso completo de limpieza y transformación de datos utilizando SQL, con el objetivo de preparar un dataset de empleados para su posterior análisis.

El trabajo se centró en identificar y corregir inconsistencias, estandarizar formatos, mejorar la calidad de los datos y generar nuevas variables que aporten valor analítico. A través de este proceso, se aplicaron distintas técnicas de limpieza como eliminación de duplicados, normalización de textos, conversión de tipos de datos y manejo de valores nulos.

El resultado final es un conjunto de datos estructurado, consistente y listo para ser utilizado en herramientas de análisis y visualización.

Contexto de la Base de Datos Original

La base de datos original contenía información de empleados en una sola tabla, incluyendo variables como identificador, nombre, apellido, fecha de nacimiento, género, área, salario, fechas laborales y tipo de contratación.

Sin embargo, durante la fase de exploración inicial se detectaron múltiples problemas que afectaban la calidad e integridad de los datos:

- **Errores en nombres de columnas,** derivados de problemas de codificación, lo que dificultaba su interpretación y manejo.
- **Presencia de registros duplicados**, lo que comprometía la precisión de cualquier análisis posterior.
- **Problemas en campos de texto**, incluyendo espacios innecesarios al inicio, al final y entre palabras, lo que afectaba la calidad y uniformidad de los datos.
- **Inconsistencias en variables categóricas,** particularmente en la columna de género, donde los valores se encontraban en español mientras que otras variables, como el área, estaban en inglés. Por esta razón, se decidió estandarizar todos los valores a un solo idioma (inglés) para mantener coherencia en el dataset.
- **Tipos de datos incorrectos,** como el salario almacenado como texto con símbolos (por ejemplo, $ y comas), lo que impedía su uso en cálculos.
- **Fechas almacenadas como texto,** con diferentes formatos (uso de "/" y "-"), lo que dificultaba su interpretación y análisis.
- **Campos con información no estandarizada,** como la columna type, que contenía valores numéricos en lugar de categorías descriptivas.
- **Presencia de valores vacíos,** representados como cadenas vacías en lugar de valores nulos.
- **Formato inconsistente en fechas con marca de tiempo,** particularmente en la columna finish_date, que incluía tanto fecha como hora en un formato no uniforme.

- # Preparación del entorno y exploración inicial

Creación y selección de la base de datos

```
create database if not exists clean;
use clean;
```

En este paso se creó una base de datos llamada clean para trabajar de manera organizada. Posteriormente, el dataset fue importado desde un archivo CSV utilizando la interfaz de MySQL Workbench. Una vez cargada la información, se seleccionó la base de datos con la instrucción use clean; para comenzar el proceso de limpieza.

Exploración inicial de los datos

```
select * from limpieza limit 10;
select * from limpieza;
```

Se realizaron consultas para visualizar los primeros registros y posteriormente el contenido completo de la tabla. Esto permitió comprender la estructura del dataset e identificar de manera preliminar posibles inconsistencias en los datos.

Creación de un procedimiento almacenado

```
delimiter //
create procedure limp()
begin
  select * from limpieza;
end//
delimiter ;
call limp();
```

Se creó un procedimiento almacenado llamado limp, el cual permite consultar rápidamente el contenido de la tabla. Esto facilitó la revisión de los datos en distintas etapas del proceso sin necesidad de repetir la misma consulta.

- ## Corrección de nombres de columnas y problemas de encoding

Renombrado de columnas con errores de codificación

```
alter table limpieza change column `ï»¿Id?empleado` Id_emp varchar (20) null;
alter table limpieza change column `gÃ©nero` Gender varchar (20) null;
```

Se corrigieron los nombres de columnas que presentaban caracteres incorrectos debido a problemas de codificación. Este ajuste permitió mejorar la legibilidad, evitar errores en consultas y establecer una nomenclatura más clara y estandarizada.

- ## Identificación y eliminación de duplicados

Detección de registros duplicados

```
select id_emp, count(*) as cantidad_duplicados
from limpieza
group by id_emp
having count(*) > 1;
```

Se agruparon los registros por el identificador del empleado (id_emp) con el fin de detectar aquellos que se encontraban repetidos dentro del dataset.

Conteo total de duplicados

```
select count(*) as cantidad_duplicados
from (
select id_emp, count(*) as cantidad_duplicados
from limpieza
group by id_emp
having count(*)>1
) as subquery;
```

Se calculó la cantidad total de registros duplicados mediante una subconsulta, lo que permitió dimensionar el impacto de este problema en la base de datos.

- # Eliminación de duplicados

Respaldo de la tabla original

```
rename table limpieza to conduplicados;
```

Se renombró la tabla original con el objetivo de conservar una copia con los registros duplicados, permitiendo trabajar sobre una versión controlada sin perder la información inicial.

Creación de tabla temporal sin duplicados

```
create temporary table Temp_limpieza as
select distinct * from conduplicados;
```

Se creó una tabla temporal que contiene únicamente registros únicos mediante el uso de SELECT DISTINCT, eliminando así los duplicados presentes en la tabla original.

Validación del proceso

```
select count(*) as original from conduplicados;
select count(*) as original from temp_limpieza;
```

Se comparó el número de registros antes y después del proceso para verificar que la eliminación de duplicados se realizó correctamente.

Creación de la tabla limpia

```
create table limpieza as select * from temp_limpieza;
call limp();
drop table conduplicados;
```

Se generó una nueva tabla permanente con los datos depurados y posteriormente se eliminó la tabla anterior que contenía duplicados, consolidando así la base de datos limpia.

- # Configuración adicional

Desactivación del modo seguro

```
set sql_safe_updates = 0;
```

Se desactivó el modo seguro de actualizaciones para permitir modificaciones en la tabla sin restricciones adicionales, lo cual facilitó la ejecución de sentencias UPDATE durante el proceso de limpieza.

- # Estandarización adicional de columnas

Renombrado de columnas

```
ALTER TABLE limpieza CHANGE COLUMN Apellido Last_Name varchar (50) null;
ALTER TABLE limpieza CHANGE COLUMN star_date start_date varchar (50) null;
```

Se ajustaron los nombres de columnas para corregir errores y mejorar la consistencia en la nomenclatura, manteniendo un formato uniforme en todo el dataset.

- # Revisión de metadatos y limpieza de campos de texto

Revisión de la estructura de la tabla

```
DESCRIBE limpieza;
```

Se consultaron los metadatos de la tabla con el fin de conocer la estructura actual, incluyendo nombres de columnas y tipos de datos. Esto permitió identificar campos que requerían ajustes adicionales durante el proceso de limpieza.

Identificación de espacios innecesarios en texto

```
CALL limp();
select name from limpieza
where length(name) - length(trim(name)) > 0;
```

Se analizaron los valores de la columna name para detectar la presencia de espacios en blanco al inicio o al final del texto, lo cual afecta la consistencia de los datos.

Limpieza de espacios en la columna name

```
select name, trim(name) as name
from limpieza
where length(name) - length(trim(name)) > 0;

update limpieza
set name = trim(name)
where length(name) - length(trim(name)) > 0;
```

Se aplicó la función TRIM() para eliminar espacios innecesarios en los valores de la columna name, mejorando la calidad y uniformidad de los datos.

Limpieza de espacios en la columna last_name

```
select last_name, trim(last_name) as last_name
from limpieza
where length(last_name) - length(trim(last_name)) > 0;

update limpieza
set last_name = trim(last_name)
where length(last_name) - length(trim(last_name)) > 0;
```

Se realizó el mismo proceso en la columna last_name, eliminando espacios en blanco al inicio y al final de los registros.

Corrección de espacios múltiples en la columna area

```
UPDATE limpieza SET area = REPLACE(area, ' ', '    ');
call limp();

select area from limpieza
where area regexp '\\s{2,}';

select  area,  trim(regexp_replace(area,  '\\s+','  '))  as
ensayo from limpieza;

update limpieza
SET area = trim(regexp_replace(area, '\\s+','  '));

call limp();
```

Se detectó la presencia de múltiples espacios entre palabras dentro de la columna area. Para corregirlo, se utilizó una expresión regular que permite reemplazar cualquier secuencia de espacios por un único espacio, logrando así una estandarización en los valores de texto.

• Estandarización de variables categóricas

Normalización de la columna gender

```
select gender,
case
 when gender = 'hombre' then 'male'
   when gender = 'mujer' then 'female'
   else 'other'
END as gender1
from limpieza;

update limpieza
set Gender = case
 when Gender = 'hombre' then 'male'
   when Gender = 'mujer' then 'female'
   else 'other'
END;
```

Se estandarizaron los valores de la columna gender, transformando los registros en español a su equivalente en inglés (male, female) y asignando el valor other a aquellos casos que no correspondían a estas categorías. Esto permitió mantener consistencia en el idioma dentro del dataset.

Transformación de la columna type

```
DESCRIBE limpieza;

alter table limpieza modify column type text;

select type,
CASE
 when type = 1 then 'Remote'
   when type = 0 then 'Hybrid'
   ELSE 'other'
END as ejemplo
from limpieza;

UPDATE limpieza
set type = CASE
 when type = 1 then 'Remote'
   when type = 0 then 'Hybrid'
   ELSE 'other'
END;

call limp();
```

Se modificó el tipo de dato de la columna type para permitir la conversión de valores numéricos a texto. Posteriormente, se transformaron los valores originales (1 y 0) en categorías descriptivas (Remote y Hybrid), mejorando la interpretabilidad de la variable.

- ## Transformación de variables numéricas

Limpieza y conversión de la columna salary

```
select salary,
CAST(trim(REPLACE(REPLACE(salary, '$',''), ',','')) AS decimal (15, 2)) AS salary1
from limpieza;
```

Limpieza y conversión de la columna salary

```
Update limpieza
set salary = CAST(trim(REPLACE(REPLACE(salary, '$',''), ',','')) AS decimal (15, 2));

alter table limpieza modify column salary int null;
describe limpieza;
```

La columna salary se encontraba almacenada como texto e incluía símbolos como $ y separadores de miles (comas), lo que impedía su uso en operaciones numéricas.

Se realizó una limpieza eliminando estos caracteres y posteriormente se convirtió el valor a un formato numérico con decimales. Finalmente, se ajustó el tipo de dato de la columna a entero (INT), permitiendo su uso en cálculos y análisis.

- ## Estandarización y conversión de fechas

Transformación de la columna birth_date

```
select birth_date, case
 when birth_date like '%/%' then date_format(str_to_date(birth_date, '%m/%d/%y'), '%Y-%m-%d')
 when birth_date like '%-%' then date_format(str_to_date(birth_date, '%m-%d-%y'), '%Y-%m-%d')
else null
end as new_birth_date
from limpieza;
update limpieza
set birth_date = case
 when birth_date like '%/%' then date_format(str_to_date(birth_date, '%m/%d/%Y'), '%Y-%m-%d')
 when birth_date like '%-%' then date_format(str_to_date(birth_date, '%m-%d-%Y'), '%Y-%m-%d')
 else null
end;
```

Transformación de la columna birth_date

```
alter table limpieza modify column birth_date date;
describe limpieza;
```

La columna birth_date presentaba formatos inconsistentes, utilizando tanto "/" como "-" como separadores. Se aplicó una transformación condicional para estandarizar todas las fechas al formato YYYY-MM-DD. Posteriormente, se cambió el tipo de dato a DATE para permitir su uso en cálculos.

Transformación de la columna start_date

```
select start_date from limpieza;

select start_date, case
 when start_date like '%/%' then date_format(str_to_date(start_date, '%m/%d/%y'), '%Y-%m-%d')
   when start_date like '%-%' then date_format(str_to_date(start_date, '%m-%d-%y'), '%Y-%m-%d')
else null
end as new_start_date
from limpieza;

update limpieza
set start_date = case
 when start_date like '%/%' then date_format(str_to_date(start_date, '%m/%d/%Y'), '%Y-%m-%d')
   when start_date like '%-%' then date_format(str_to_date(start_date, '%m-%d-%Y'), '%Y-%m-%d')
   else null
end;

call limp();

alter table limpieza modify column start_date date;
describe limpieza;
```

Transformación de la columna start_date

Se aplicó el mismo proceso de estandarización en la columna start_date, asegurando un formato uniforme y convirtiendo el tipo de dato a DATE.

Transformación de la columna finish_date

```
select finish_date from limpieza;

SELECT  finish_date,  str_to_date(finish_date,  '%Y-%m-%d  %H:%i:%s')
AS fecha FROM limpieza;

alter table limpieza add column date_backup text;
update limpieza set date_backup = finish_date;

UPDATE limpieza
set finish_date = str_to_date(finish_date, '%Y-%m-%d %H:%i:%s UTC')
WHERE finish_date <> '';
```

La columna finish_date contenía valores con formato de fecha y hora (timestamp). Antes de realizar cualquier transformación, se creó una copia de respaldo para evitar pérdida de información. Posteriormente, se convirtió el contenido a un formato de fecha válido.

Separación de fecha y hora

```
ALTER TABLE limpieza
   ADD COLUMN fecha date,
   ADD COLUMN hora time;

UPDATE limpieza
SET fecha = date(finish_date),
   hora = time(finish_date)
where finish_date is not null and finish_date <> '';
```

Se crearon columnas adicionales para separar la fecha y la hora, lo que permite un análisis más detallado en caso de ser necesario.

Estandarización final de finish_date

```
update limpieza set finish_date = null where finish_date = '';

alter table limpieza modify column finish_date datetime;
describe limpieza;
```

Se reemplazaron los valores vacíos por NULL y se ajustó el tipo de dato de la columna a DATETIME, asegurando consistencia en el formato.

• Creación de nuevas variables

Cálculo de la edad

```
alter table limpieza add column age int;

select name, birth_date, start_date,
timestampdiff(year, birth_date, start_date) as edad_de_ingreso
from limpieza;

UPDATE limpieza
set AGE = timestampdiff(year, birth_date, curdate());

call limp();
```

Se creó la columna age, calculando la edad de cada empleado a partir de su fecha de nacimiento y la fecha actual, utilizando la función TIMESTAMPDIFF.

Generación de correos electrónicos

```
Select concat(substring_index(name,' ', 1), '_',
substring(last_name, 1, 2), '.',
substring(type, 1, 1), '@consulting.com') as email
from limpieza;
```

Generación de correos electrónicos

```
alter table limpieza add column email varchar(100);
update limpieza
set email = concat(substring_index(name,' ', 1), '_',
substring(last_name, 1, 2), '.',
substring(type, 1, 1), '@consulting.com');
```

Se generó una nueva columna email a partir del nombre, apellido y tipo de trabajo del empleado, creando identificadores únicos útiles para análisis o integración con otros sistemas.

• Generación del dataset final

Generación de correos electrónicos

```
SELECT Id_emp, Name, Last_name, age, gender, area, type, salary,
email, finish_date
FROM LIMPIEZA
WHERE finish_date <= curdate() or finish_date is null
order by area, Last_name;
```

Finalmente, se seleccionaron las columnas relevantes para conformar el dataset limpio, aplicando un filtro para considerar únicamente registros vigentes o sin fecha de finalización.

El resultado es una base de datos estructurada, consistente y preparada para su uso en procesos de análisis y visualización.

Conclusiones y aprendizajes

El proceso de limpieza y transformación de datos permitió convertir un dataset con múltiples inconsistencias en una base de datos estructurada, confiable y lista para su análisis. A lo largo del proyecto se abordaron problemas comunes en el manejo de datos, como duplicados, formatos incorrectos, valores inconsistentes y errores en los tipos de datos.

Uno de los principales aprendizajes fue la importancia de realizar una exploración inicial detallada, ya que esto permite identificar de manera clara los problemas que deben resolverse antes de iniciar cualquier análisis. Asimismo, se reforzó la comprensión del proceso de limpieza de datos y su impacto en la calidad de la información.

También se destacó la relevancia de estandarizar la información, tanto en formato como en idioma, para garantizar coherencia en el dataset. La transformación de variables categóricas y la conversión de datos a tipos adecuados facilitaron la interpretación y el uso posterior de la información.

Adicionalmente, la creación de nuevas variables, como la edad y el correo electrónico, demostró cómo es posible enriquecer un dataset y prepararlo para análisis posteriores.

Finalmente, este proyecto permitió consolidar habilidades fundamentales en SQL orientadas a la preparación de datos, una etapa clave dentro del flujo de trabajo de análisis, ya que la calidad de los resultados depende directamente de la calidad de los datos utilizados.

Contáctame

Soy Analista de Datos Junior con enfoque en visualización y análisis exploratorio de datos. Me interesa seguir desarrollando proyectos que transformen información en insights claros y accionables. Si deseas colaborar o conectar para hablar de análisis de datos, estaré encantada de conversar.

Fanny Aguila Tlecuitl

Email
fannytlecuitl@gmail.com

GitHub
https://github.com/Fannn75

LinkedIn
https://www.linkedin.com/in/fanny-tlecuitl/